# LLMs as Design Partners for AI-Based System Patterns: An Empirical Evaluation

Felipe Rodrigues de Oliveira
State University of Ceara, Brazil
Fortaleza, Ceara, Brazil
felipeo.rodrigues@aluno.uece.br

Felipe Vasconcelos De Souza
State University of Ceara, Brazil
Fortaleza, Ceara, Brazil
felipevasc.souza@aluno.uece.br

Ana Luiza Bessa De Paula Barros
State University of Ceara, Brazil
Fortaleza, Ceara, Brazil
analuiza.barros@uece.br

Paulo Henrique Maia
State University of Ceara, Brazil
Fortaleza, Ceara, Brazil
pauloh.maia@uece.br

## Abstract

The growing integration of Artificial Intelligence (AI) components into software systems challenges engineers to design architectures that combine learning, reasoning, and reliability. This paper presents an empirical evaluation of how Large Language Models (LLMs) can assist in designing AI-based systems through pattern generation. We analyzed 65 documented design patterns using three state-of-the-art LLMs—Gemini 1.5 Flash, Claude 3.5 Sonnet, and o1-mini—applying a mixed-method framework that combines quantitative similarity analysis, expert qualitative judgment, and blind human evaluation. Results show that LLMs generated solutions equivalent or superior to literature-based ones for most patterns, revealing distinct model strengths: *o1-mini* achieved the highest alignment with existing designs, while *Claude 3.5 Sonnet* received the strongest human preference. The study contributes a replicable methodology for assessing LLM reasoning in software design and highlights the potential of LLMs as co-creators that complement human creativity in the engineering of AI-based systems.

## Keywords

LLM, Design Patterns, AI-based systems

**ACM Reference Format:**
Felipe Rodrigues de Oliveira, Felipe Vasconcelos De Souza, Ana Luiza Bessa De Paula Barros, and Paulo Henrique Maia. 2026. LLMs as Design Partners for AI-Based System Patterns: An Empirical Evaluation. In *2026 IEEE/ACM 5th International Conference on AI Engineering - Software Engineering for AI (CAIN '26), April 12–13, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3793653.3793760

## 1 Introduction

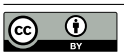
Artificial intelligence (AI) is increasingly embedded in modern software systems, driving innovation across diverse domains such as healthcare, finance, manufacturing, and autonomous vehicles [6]. Systems that incorporate AI components are known as *AI-based systems* [12]. Unlike traditional software, whose behaviour is explicitly programmed, the behaviour of AI-based systems emerges from models trained on data. This shift introduces new design and engineering challenges that go beyond conventional software practices.

Although AI-based systems remain software systems at their core, and can therefore benefit from established software engineering principles such as modularity, separation of concerns, and agile development [1], they also demand specialized practices. These include managing uncertainty, handling continuous learning and retraining, and ensuring interpretability, fairness, and security. The combination of traditional and data-driven concerns has given rise to the discipline of *AI Engineering* [3], which extends software engineering by incorporating concepts, methods, and technologies from AI and data science.

A widely adopted means of capturing recurring solutions in software design is through *design patterns* [17]. Each pattern represents an abstract reusable solution to a common design problem, providing both conceptual guidance and implementation flexibility, and fostering reusability and communication across interdisciplinary teams. In the context of AI-based systems, design patterns help address typical integration challenges such as data pipeline orchestration, model deployment, retraining, interpretability, and reliability [9].

The growing complexity of AI-based software development has prompted interest in how automated tools, particularly Large Language Models (LLMs), can assist in design reasoning. LLMs have demonstrated remarkable capabilities in code generation, documentation, and problem solving across a wide spectrum of software engineering activities [7, 13]. However, while previous studies have demonstrated that LLMs can generate code [12] and even recognize classical software design patterns in source code [14], their capacity for higher-level design reasoning remains largely unexplored [15]. Here, higher-level design reasoning refers to reasoning about architectural intent, component interactions, and system-level decisions, rather than the generation of concrete implementations or syntactic code structures.

This paper investigates a central research question: **to what extent can LLMs discover or generate design pattern solutions for AI-based systems as effectively as humans?** Understanding this question is critical for both software engineering research and industry practice, especially as AI engineering emerges to bridge traditional software design with machine-learning–driven systems [9]. If current LLMs can reason about design patterns at a conceptual



This work is licensed under a Creative Commons Attribution 4.0 International License.
*CAIN '26, Rio de Janeiro, Brazil*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2475-6/26/04
https://doi.org/10.1145/3793653.3793760

level, they could transform the way software design knowledge is represented, retrieved, and applied, potentially moving beyond static pattern repositories toward more dynamic AI-driven design knowledge bases [11]. Achieving such a capability would elevate LLMs from mere code assistants to genuine *design partners*, i.e., agents capable of suggesting, comparing, and adapting architectural solutions in collaboration with human engineers. Such progress could have major implications for AI Engineering, where the design of AI-based systems must integrate heterogeneous components and deal with nondeterministic outcomes [11], while also reconciling human and machine decision-making processes in complex architectures [9]. By investigating this capability empirically, the present study not only evaluates the current performance of state-of-the-art models but also lays the groundwork for understanding how future AI systems might contribute to software design as a fundamentally cognitive and creative process.

Addressing this question requires evaluating not only the similarity of LLM-generated design patterns to those documented in the literature but also their perceived quality and usefulness from a human perspective. To this end, we evaluate three state-of-the-art LLMs—*Gemini 1.5 Flash*, *Claude 3.5 Sonnet*, and *o1-mini*—across 65 design patterns identified in the AI Patterns Repository [9]. The evaluation combines quantitative similarity assessment, qualitative expert analysis, and blind human judgment, forming a multi-dimensional methodology that captures both objective and perceptual aspects of design reasoning. In this study, equivalence refers to semantic alignment with literature-based solutions as assessed through similarity analysis; effectiveness refers to conceptual adequacy confirmed by expert judgment in cases of low similarity; and superiority is discussed exclusively in terms of practitioner preference observed in the blind evaluation.

The contributions of this paper are threefold: (i) a novel, replicable framework that integrates quantitative metrics and human judgment to assess the design capabilities of LLMs; (ii) an empirical comparison of three leading LLMs on 65 AI-based design patterns, revealing complementary strengths and tendencies across models; (iii) an empirical evidence that LLM-generated designs can achieve a level of quality comparable to, and in many cases preferred over, human-authored solutions.

The remainder of this paper is organized as follows: Section 2 presents the background on AI-based design patterns, while Section 3 reviews the main related work. Section 4 details the research methodology. Results are reported in Section 5 and discussed in depth in Section 6. Threats to validity are shown in Section 7. Finally, Section 8 draws conclusions and future work.

## 2 Design Patterns for AI-based Systems

The dataset used in this study was curated by Heiland et al. [9] and is publicly available through the AI Patterns Repository [8]. It comprises 65 design patterns for AI-based systems, each documented with structured fields such as *name*, *type* (Traditional or New), *category*, *motivation*, and *solution*. This repository serves as a comprehensive reference for researchers and practitioners, consolidating both classical software-engineering patterns adapted to AI contexts and novel patterns that have emerged from AI-specific challenges. This dataset was selected because it provides a systematically curated and well-documented collection of AI-specific design patterns, developed with explicit research rigor and a focus on architectural and design-level concerns.

Heiland et al. [9] identified a total of 70 patterns, of which 36 represent traditional software engineering patterns reinterpreted for AI systems, while 34 are entirely new. The patterns span several categories—*architecture*, *deployment*, *implementation*, *process*, *testing & quality assurance*, *topology*, and *security & safety*. Among these, architectural and deployment patterns are the most prevalent, reflecting the central role of data pipelines, model training, inference, and monitoring in modern AI-based architectures.

Examples of traditional patterns adapted to AI contexts include the *Multi-Layer Pattern* and the *Strategy Pattern*, which address modularity and model interchangeability, respectively. In contrast, new patterns such as *Workflow Pipeline* and *Two-Phase Predictions* target challenges introduced by learning components, such as scalable training and efficient inference. Other patterns, like *Encapsulating ML Models within Rule-based Safeguards* and *Ethics Credentials*, explicitly incorporate concerns of safety, trustworthiness, and ethical compliance, which are increasingly essential in AI engineering.

Table 1 illustrates representative examples, showing how patterns encapsulate typical design problems and their associated solutions. For instance, the *AI Mode Switcher* addresses runtime flexibility by allowing an AI component to be deactivated when risk assessment becomes uncertain, while the *AI Pipelines* pattern divides a complex inference task into smaller, composable steps handled by specialized models. Such examples demonstrate how design patterns for AI-based systems bridge well-established software engineering principles with emerging data-centric practices.

## 3 Related Work

Several recent studies have explored how Large Language Models can be applied to software design and architecture tasks, with particular attention to their ability to recognise or implement design patterns. However, most of these works focus on code-level reasoning rather than conceptual design generation—the focus of our study.

Pan et al. [14] conducted an empirical evaluation of *Code LLMs* to assess their understanding of software design patterns. Their framework evaluated recognition, generation, and comprehension of twelve classical patterns (e.g., *Singleton*, *Factory*) across curated Python and Java repositories. They found that although LLMs are versatile in various code-related tasks, they frequently misunderstand or incorrectly implement established design patterns, leading to structural inconsistencies and potential quality degradation.

Schindler and Rausch [16] investigated the automatic detection of design patterns in complex codebases, an area where traditional static analysis tools often fail due to implementation variability. They used LLMs, specifically ChatGPT-3.5 and 4, to identify instances of the *Composite* pattern by analysing the roles that classes play within the pattern's structure. Results showed that LLMs could partially detect these patterns, though accuracy varied across roles and models, with GPT-4 exhibiting fewer hallucinations and higher precision.

**Table 1: Design Patterns for AI-based Systems Examples**

| IA Design Pattern | Problem (motivation) | Solution | Category | Type |
|---|---|---|---|---|
| AI Mode Switcher | AI is often complex and hard to explain, thus making detailed risk assessment difficult. Adopting AI can be considered as a major architectural design decision when designing a system. | An AI component is designed to be flexibly switched off at run-time. | Security & Safety | New |
| AI Pipelines | Complex prediction or synthesis use cases are often difficult to accomplish with a single AI tool or model. | Divide the problem into smaller consecutive steps, then combine several existing AI tools or custom models into an inference-time AI pipeline, where each specialized tool or model is responsible for a single step. | Architecture; Topology | New |

Pandey et al. [15] also evaluated the ability of LLMs to identify design patterns in source code, showing that even models without explicit pre-training on this task can capture semantic and syntactic information useful for *Design Pattern Recognition* (DPR). Their results indicate that LLM-based DPR tools can outperform traditional feature-based approaches by leveraging deep contextual understanding.

Jánki and Bilicki [10] proposed a hybrid approach combining rule-based and prompt-based reasoning for recognizing architectural patterns in Angular web applications. They formalized 30 rules describing the characteristics of MVC, MVP, and MVVM architectures and encoded these as GPT-4 prompts. Applied to real-world repositories, their method achieved approximately 90% classification accuracy—outperforming traditional static tools (approximately 80%)—demonstrating that LLMs, when properly guided, can reliably distinguish overlapping architectural styles.

Kim [11] examined how well LLMs can *apply* design patterns through code refactoring. Using the Role-Based Meta-Modeling Language (RBML) to formalize pattern specifications, the author instructed ChatGPT-4 to refactor code according to given patterns. Verification against structural constraints yielded 98% pattern conformance and 87% completeness across three case studies, suggesting that LLMs can effectively enforce structural design principles when supplied with explicit formalizations.

Taken together, these studies demonstrate the growing interest in understanding LLMs' capabilities for software design, though they primarily emphasize recognition, application, or refactoring rather than conceptual reasoning. In contrast, our work evaluates whether LLMs can *generate* pattern-based solutions for AI-oriented design problems that are comparable to, and in some cases preferred over, human-authored solutions, operating at the level of design intent.

## 4 Methodology

This section presents the methodology adopted in this study, including the research questions that guided the evaluation and the procedures followed to address them.

### 4.1 Dataset and Selected LLMs

The experimental dataset comprises 65 design patterns collected from Heiland et al. [9], which originally documented 70 patterns. Five patterns (*Adapter Pattern*, *Builder Pattern*, *Data flows up*, *Model flows down*, and *Decorator Pattern*) were excluded because they lacked a "Motivation" field—an essential input for prompt generation.

Three advanced LLMs were selected for evaluation: *Google Gemini 1.5 Flash*, *Anthropic Claude 3.5 Sonnet*, and *OpenAI o1-mini*. They were chosen for their complementary characteristics: Gemini 1.5 Flash for efficiency and extended context window, Claude 3.5 Sonnet for interpretive precision and reasoning accuracy, and o1-mini for its strong performance in structured problem-solving. These models represent three distinct families of state-of-the-art general-purpose LLMs, ensuring diversity and balance in the comparative evaluation. The models were selected to support a comparative analysis of LLM behavior in design-oriented tasks.

### 4.2 Research Questions

The overarching goal of this study is to investigate whether LLMs can reason about and generate design-pattern solutions for AI-based systems in a way comparable to (or even exceeding) human-authored ones. To operationalise this goal, we formulated four research questions, each addressing a distinct dimension of capability and evaluation.

**RQ1 – To what extent do the solutions proposed by LLMs resemble those documented in the literature?** *Rationale:* This question assesses how closely model-generated solutions align with established design knowledge. It serves as a baseline for understanding whether LLMs reproduce canonical solutions or diverge from human practices.

**RQ2 – How do human experts evaluate LLM-generated solutions that differ from those found in the literature?** *Rationale:* A lack of textual similarity does not necessarily imply poor quality. This question examines whether alternative solutions proposed by LLMs are still functionally correct or valuable from an expert's perspective.

**RQ3 – Which solutions are preferred by practitioners: those documented in the literature or those generated by LLMs?** *Rationale:* Beyond objective similarity, perceived usefulness matters. This question explores human preference and acceptance, revealing whether practitioners find LLM-generated solutions more understandable, creative, or effective.

**RQ4 – Which LLM achieves the best overall performance across similarity, expert judgment, and practitioner preference?** *Rationale:* Each model family has distinct strengths. This question integrates all evidence to determine which model provides

the most balanced performance and to what extent these models complement one another.

## 4.3 Research Design

Figure 1 illustrates the three-phase methodology adopted in this work: (i) **Preparation**, which involved generating LLM-based solutions for the selected design patterns; (ii) **Similarity Assessment**, which measured how closely these generated solutions aligned with those documented in the literature; and (iii) **Qualitative Analysis**, which captured human perception and expert judgment regarding solution effectiveness.

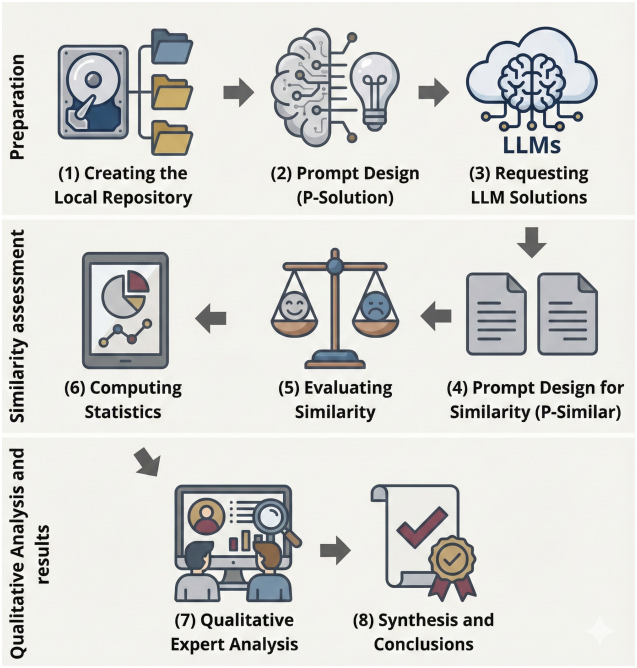



**Figure 1: Overview of the Methodology**

Each phase comprised a structured set of steps:

(1) **Creating the Local Repository:** The 65 AI design patterns were extracted from [9] and stored in a local CSV file containing the most relevant fields—*name*, *category*, *motivation*, and *solution*. This ensured consistent access and traceability for all subsequent comparisons.
(2) **Prompt Design (*P-Solution*):** A prompt was engineered to request that each model generate a solution to a given design pattern problem. The prompt explicitly incorporated contextual elements (pattern name and motivation) and constrained the output to a single, concise paragraph resembling the format of the original dataset. This design encouraged focused, comparable answers aligned with the expected structure of pattern documentation.
(3) **Requesting LLM Solutions:** All 65 pattern problems were automatically submitted to each LLM through API calls. Each model generated independent solutions, which were stored in a unified CSV file for reproducibility and later analysis.
(4) **Prompt Design for Similarity (*P-Similar*):** A second prompt was written to evaluate the similarity between literature-based and LLM-generated solutions. It included the pattern's name, motivation, both candidate solutions, and requested a numeric rating from 1 to 5 on a Likert-based scale (1 = not similar at all; 5 = almost identical). This standardized scoring mechanism reduced subjectivity while preserving interpretability.
(5) **Evaluating Similarity:** Each LLM was then asked to assess similarity between the literature solution and all generated solutions, including those from the other models. Thus, LLMs served both as *solution generators* and *cross-evaluators*. This cross-model scoring mitigated self-bias and provided a balanced quantitative measure of alignment with human-authored designs.
(6) **Computing Statistics:** Descriptive statistics—including mean, median, standard deviation, and range—were computed to characterize model performance across all patterns. These metrics enabled comparison of central tendencies and variability in model behavior.
(7) **Qualitative Expert Analysis:** For cases where LLM similarity scores were $\leq 3$, three human experts in software engineering and AI (each with more than 15 years of experience) qualitatively assessed whether the LLM-generated solutions effectively solved the documented problem. This analysis complemented the quantitative phase, providing human insight into conceptual adequacy.
(8) **Synthesis and Conclusions:** The final step consolidated the quantitative and qualitative findings to assess the overall potential of LLMs in identifying and formulating AI design-pattern solutions.

## 4.4 Prompt-Engineering Principles

The prompts used in this study were carefully designed following established prompt engineering strategies to maximize reliability, transparency, and comparability of outputs. Two complementary principles were adopted:

- **Prompt Decomposition:** Instead of using a single monolithic prompt, the task was divided into two independent stages—generation (*P-Solution*) and evaluation (*P-Similar*). This decomposition minimized ambiguity, avoided prompt contamination between tasks, and ensured that each model processed inputs with a clearly defined goal.
- **Role Instruction and Anchoring:** Each prompt explicitly framed the model's role (e.g., "act as a software design assistant") and provided semantic anchors such as the pattern name and motivation. Few-shot examples were included to guide output style, while concise length constraints promoted consistency across responses.

Both prompts are reproduced in Table 2. For *P-Solution*, a single-paragraph answer was required to mirror the structure of the original repository. For *P-Similar*, only a numeric score was permitted, ensuring a standardized, machine-readable similarity scale.

**Table 2: Prompts used in the study.**

| Prompt |
|---|
| **[P-Solution]**: Act as a software design assistant. Based on the following problem: "${motivation}" Please provide a solution in a format that resembles a design pattern, similar to the example below. Each answer should be concise, focusing on key components that address the issue effectively.<br>Example: Solution: Ensure that only one instance of the class is created and accessible throughout the program's execution. Implement a static method to create this single instance on the first call and return the same instance on subsequent calls.<br>Provide the solution summarized in just one paragraph. |
| **[P-Similar]**: Consider the problem in AI-based systems:<br>Name: ${name}. Problem: ${motivation}.<br>Two solutions are provided below. Act as a software design assistant and compare their similarity using the Likert scale (1 to 5), where: 1 - Not at all similar 2 - Slightly similar 3 - Moderately similar 4 - Very similar 5 - Almost identical.<br>— Solution 1: ${solution} — Solution 2: ${solutionLLM} — Provide only the similarity score (1-5) as your response. |

## 4.5 Justification for LLM Self-Evaluation

Allowing LLMs to act as evaluators introduces potential bias; however, several precautions were taken to reduce circularity. First, each model evaluated all generated solutions—including those produced by other models—without access to author or model identifiers. This cross-model design prevented self-referencing and distributed judgment across three independent evaluators. Second, the final interpretation relied not solely on LLM ratings but also on expert and practitioner validation (RQs 2 and 3). This triangulation of automated and human perspectives increases confidence in the robustness of the results. Importantly, similarity evaluations were performed using independent prompts, and the evaluating models were not informed about the origin of the generated solutions, further reducing the risk of self-evaluation bias.

In summary, the methodology combines controlled prompt design, quantitative comparison, and human verification to provide a balanced, reproducible assessment of how effectively LLMs can propose and evaluate design solutions for AI-based systems.

# 5 Results

This section presents the results of the empirical evaluation. Each research question (RQ) is addressed in a dedicated subsection.

## 5.1 RQ1 – To what extent do LLM-generated solutions resemble those documented in the literature?

To compare the similarity between the 195 LLM-generated solutions (65 patterns × 3 models) and the 65 literature-based ones [9], we first tested some automatic text-similarity metrics. The *BERTScore* [18], a widely used method in text-generation evaluation, was initially applied. However, it proved inadequate for this task: the metric produced numerous false positives, rating conceptually opposite solutions as highly similar. For instance, when evaluating the *Synchronous Pattern*, the *BERTScore* reported an F1 = 0.85 for an asynchronous solution generated by Gemini 1.5 Flash—clearly an incorrect alignment. This limitation echoes previous studies [4] that note that lexical overlap metrics (Word2Vec, FastText, or BERT-based) cannot reliably capture coherence or domain-specific logic.

To address this, we adopted a refined comparison strategy based on a 5-point Likert scale, inspired by Bojić et al. [2], where similarity is judged semantically rather than lexically. Each of the three LLMs independently evaluated all generated solutions (including those from other models) against the literature reference. This yielded a set of human-interpretable similarity scores (1 = Not at all similar; 5 = Almost identical).

Figure 3 summarizes the distribution of similarity scores. All evaluators showed median values around 3 (moderate similarity) but with subtle differences in spread. Claude 3.5 Sonnet exhibited the most conservative behavior (mean = 3.08, SD = 0.78), Gemini 1.5 Flash tended to assign slightly higher scores (mean = 3.38, SD = 0.83), and o1-mini displayed the widest range (mean = 3.25, SD = 1.15). When analyzed as evaluated models, o1-mini achieved the highest alignment with the literature (mean = 3.44, median = 4), followed by Claude 3.5 Sonnet (mean = 3.35) and Gemini 1.5 Flash (mean = 3.31).

Across all patterns, 41 out of 65 (63.07%) scored above 3, indicating moderate to strong similarity; four patterns (6.5%) were neutral (score = 3), and twenty (30.77%) were below 3, as illustrated by Table 3. These results demonstrate that LLMs frequently reproduce literature-consistent solutions. Patterns such as *Condition-based Serving*, *Data Lake (for ML)*, *Deploy Canary Model*, *Infrastructure as Code*, and *Prediction Cache Pattern* achieved very high similarity (> 4.5), whereas *Giveaway* and *Synchronous Pattern* showed the largest discrepancies.

Figure 2 depicts average similarity by pattern category. The *Topology* category achieved the highest mean score, while *Process* showed the lowest, indicating that LLMs capture architectural relationships more consistently than procedural workflows.

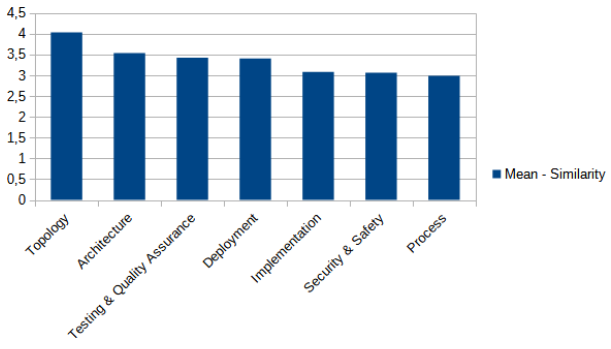



**Figure 2: Average similarity scores by pattern category.**

An additional analysis considered pattern type. "New" AI-specific patterns achieved an average similarity of 3.51, compared to 3.21 for traditional ones, suggesting that LLMs may be better attuned to novel, data-driven paradigms than to legacy software patterns.

**Table 3: Statistical Results for Each Design Pattern for AI-Based Systems**

| IA Design Pattern | Category | Type | Mean | Median | Max | Min | StdDev |
|---|---|---|---|---|---|---|---|
| Condition-based-serving pattern | Deployment | New | 4.67 | 5 | 5 | 4 | 0.50 |
| Data Lake (for ML) | Architecture | New | 4.67 | 5 | 5 | 4 | 0.50 |
| Deploy canary model | Deployment | Traditional | 4.67 | 5 | 5 | 4 | 0.50 |
| Infrastructure as Code | Process | Traditional | 4.56 | 5 | 5 | 4 | 0.53 |
| Prediction cache pattern | Architecture | Traditional | 4.56 | 5 | 5 | 4 | 0.53 |
| Prep-pred pattern | Deployment | New | 4.44 | 4 | 5 | 4 | 0.53 |
| AI Pipelines | Architecture; Topology | New | 4.22 | 4 | 5 | 4 | 0.44 |
| Proxy Pattern | Architecture | Traditional | 4.22 | 4 | 5 | 4 | 0.44 |
| Web single pattern | Deployment | New | 4.22 | 4 | 5 | 4 | 0.44 |
| Batch Serving | Deployment | Traditional | 4.11 | 4 | 5 | 3 | 0.60 |
| Lambda Architecture Pattern | Architecture | Traditional | 4.11 | 4 | 5 | 4 | 0.33 |
| Mediator Pattern | Architecture | Traditional | 4.11 | 4 | 5 | 3 | 0.60 |
| Microservice horizontal pattern | Architecture | New | 4.11 | 4 | 5 | 4 | 0.33 |
| Workflow Pipeline | Architecture; Topology | New | 4.11 | 4 | 5 | 4 | 0.33 |
| Continued Model Evaluation | Testing & Quality Assurance | New | 4.00 | 4 | 5 | 3 | 0.50 |
| Continuous Integration and Deployment | Process | Traditional | 4.00 | 4 | 4 | 4 | 0.00 |
| Functional-Style Architecture | Architecture | Traditional | 4.00 | 4 | 5 | 3 | 0.50 |
| Multi-Layer Pattern | Architecture | Traditional | 4.00 | 4 | 5 | 2 | 1.00 |
| Strategy Pattern | Implementation | Traditional | 4.00 | 4 | 5 | 3 | 0.71 |
| Alarms And Notifications On Processes | Testing & Quality Assurance | New | 3.89 | 4 | 4 | 3 | 0.33 |
| Daisy Architecture | Architecture | New | 3.89 | 4 | 4 | 3 | 0.33 |
| Two-Phase Predictions | Deployment | New | 3.89 | 4 | 4 | 3 | 0.33 |
| Data cache pattern | Architecture | Traditional | 3.78 | 4 | 4 | 3 | 0.44 |
| Data Warehouse for ML | Architecture | New | 3.78 | 4 | 4 | 3 | 0.44 |
| Fluid Architecture | Architecture; Topology | New | 3.78 | 4 | 4 | 3 | 0.44 |
| Model-load pattern | Deployment | New | 3.78 | 4 | 4 | 3 | 0.44 |
| Asynchronous pattern | Deployment; Architecture | Traditional | 3.67 | 4 | 5 | 2 | 0.87 |
| Encapsulating ML Models within Rule-based Safeguards | Security & Safety | New | 3.67 | 4 | 4 | 3 | 0.50 |
| Ethics credentials | Security & Safety | New | 3.67 | 4 | 4 | 3 | 0.50 |
| Parameter-based serving pattern | Implementation | New | 3.67 | 4 | 4 | 3 | 0.50 |
| Serving template pattern | Deployment | Traditional | 3.56 | 4 | 4 | 3 | 0.53 |
| Dedicated Model API | Deployment | New | 3.44 | 3 | 4 | 3 | 0.53 |
| Handshake or Hand Buzzer | Architecture | Traditional | 3.44 | 3 | 4 | 3 | 0.53 |
| Microservice vertical pattern | Architecture | New | 3.44 | 3 | 4 | 3 | 0.53 |
| Audit Black Box | Security & Safety | New | 3.33 | 3 | 4 | 3 | 0.50 |
| Delegation of Safety Responsibility | Security & Safety | New | 3.33 | 3 | 4 | 2 | 0.71 |
| Model-View-Controller (MVC) | Architecture | Traditional | 3.33 | 3 | 4 | 3 | 0.50 |
| Replay | Testing & Quality Assurance | Traditional | 3.33 | 3 | 4 | 3 | 0.50 |
| Circuit break pattern | Testing & Quality Assurance | Traditional | 3.22 | 3 | 4 | 2 | 0.67 |
| Secure Virtual Premise (SVP) | Security & Safety | New | 3.22 | 3 | 4 | 3 | 0.44 |
| Distinguish Business Logic from ML Model | Architecture | New | 3.11 | 3 | 4 | 2 | 0.78 |
| Internal Feedback | Process | Traditional | 3.00 | 3 | 4 | 2 | 0.50 |
| Legacy | Process | Traditional | 3.00 | 3 | 4 | 2 | 0.71 |
| Model Published as Data | Deployment | New | 3.00 | 3 | 4 | 2 | 0.87 |
| Parameter-Server Abstraction | Architecture | New | 3.00 | 3 | 4 | 2 | 0.50 |
| Client-Server | Architecture | Traditional | 2.89 | 3 | 3 | 2 | 0.33 |
| Decision Mode Switcher | Security & Safety | New | 2.89 | 3 | 4 | 2 | 0.60 |
| Closed Loop intelligence | Implementation | New | 2.67 | 3 | 3 | 2 | 0.50 |
| End-To-End Tests | Testing & Quality Assurance | Traditional | 2.67 | 3 | 3 | 2 | 0.50 |
| Stateless Serving Function | Deployment | Traditional | 2.67 | 3 | 3 | 2 | 0.50 |
| Chop Shop | Process | Traditional | 2.56 | 3 | 3 | 2 | 0.53 |
| Full Acceptance of Safety Responsibility | Security & Safety | New | 2.56 | 3 | 3 | 2 | 0.53 |
| Kappa Architecture Pattern | Architecture | Traditional | 2.56 | 2 | 4 | 2 | 0.73 |
| Model-in-image pattern | Deployment | New | 2.56 | 3 | 3 | 2 | 0.53 |
| Shadow | Process | Traditional | 2.56 | 3 | 3 | 2 | 0.53 |
| AI Mode Switcher | Security & Safety | New | 2.44 | 2 | 3 | 2 | 0.53 |
| Discard PoC code | Process | Traditional | 2.44 | 2 | 3 | 2 | 0.53 |
| Partial Rejection of Safety Responsibility | Security & Safety | New | 2.44 | 2 | 3 | 2 | 0.53 |
| Offline Predictions | Deployment | New | 2.33 | 2 | 3 | 2 | 0.50 |
| Pipes and Filters | Architecture | Traditional | 2.33 | 2 | 3 | 2 | 0.50 |
| Model Embedded in Application | Deployment | New | 2.22 | 2 | 3 | 2 | 0.44 |
| Tar pit | Architecture | Traditional | 2.11 | 2 | 3 | 2 | 0.33 |
| Player-Role Pattern | Implementation | Traditional | 2.00 | 2 | 3 | 1 | 0.71 |
| Giveaway | Process | Traditional | 1.78 | 2 | 2 | 1 | 0.44 |
| Synchronous pattern | Deployment; Architecture | Traditional | 1.22 | 1 | 2 | 1 | 0.44 |

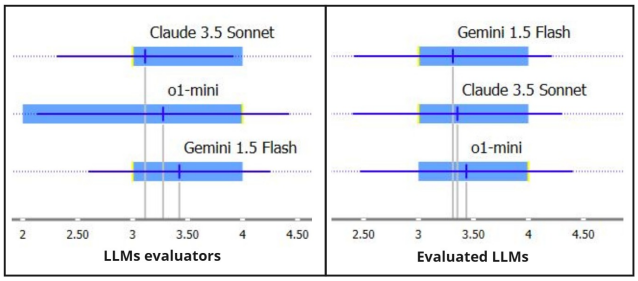



**Figure 3: Distribution of similarity scores by LLM evaluators and evaluated LLMs.**

## 5.2 RQ2 – How do human experts assess LLM solutions that differ from literature-based ones?

Low similarity does not necessarily indicate that a model produced an incorrect or ineffective solution. To explore this, three experts in software engineering and AI, each with over 15 years of experience, independently reviewed the 24 patterns rated as dissimilar (similarity $\leq 3$). Each expert judged whether the LLM-generated solutions effectively solved the problem (*Solved* = 1, *Not Solved* = 0). A solution was considered effective when at least two of the three evaluators agreed (*Solved* $\geq$ *0.67*).

As summarized in Table 5, 18 out of 24 (75 %) low-similarity cases were still deemed successful. For example, in the *Parameter-Server Abstraction* pattern, for which the literature recommends distributing data and workload over worker nodes, Claude 3.5 Sonnet and o1-mini proposed a more general *Distributed Learning Abstraction Layer*, while Gemini 1.5 Flash proposed a federated-learning variant. As detailed in Table 4, the LLMs' solutions for this pattern illustrate these alternative abstractions, all of which experts agreed effectively addressed the same underlying problem.

In total, combining RQ1 and RQ2, LLMs produced valid or equivalent solutions for 59 (41+18) out of 65 patterns (90.76%). This demonstrates that, even when textual similarity is low, conceptual adequacy often remains high.

## 5.3 RQ3 – Which solutions do practitioners consider better: literature-based or LLM-generated?

To capture practitioner perception, a blind human evaluation was conducted using a sample of 18 patterns ($\approx$ 27.7 % of the total). Patterns were selected via stratified random sampling to ensure proportional representation of both "Similar" (mean > 3) and "Not Similar" (mean $\leq$ 3) groups. The definition of the sample size ($n = 18$) represents a balance between statistical rigor and the feasibility of an in-depth human evaluation. While a formal calculation (using Cochran's formula with finite population correction) for a 5% margin of error would indicate a larger sample ($n = 56$), such a volume was considered impractical as it would impose an excessive cognitive burden on the evaluators [5]. The chosen sample of 18 patterns, in contrast, allows for analysis with 95% confidence and a 19.8% margin of error, a trade-off deemed appropriate given the exploratory nature of this research.

**Table 4: Comparison of literature and LLM-generated solutions for the Parameter-Server Abstraction pattern**

| |
|---|
| **Pattern:** Parameter-Server Abstraction |
| **Motivation:** Abstractions for distribute learning are lacking |
| **Literature Solution:** Distribute data and workload over worker nodes. Server nodes has globally shared parameters. |
| **Claude 3.5 Sonnet Solution:** Develop a Distributed Learning Abstraction Layer (DLAL) that provides a unified interface for various distributed learning frameworks and protocols. This layer should abstract away the complexities of data distribution, synchronization, and communication across nodes, offering high-level APIs for common distributed learning tasks. Implement adapters for popular frameworks, ensuring interoperability and easy integration. Include built-in support for fault tolerance, load balancing, and scalability. Provide extensible components for custom algorithms and data handling, allowing researchers and developers to focus on model development rather than distributed system intricacies. |
| **o1-mini Solution:** Implement a Distributed Learning Abstraction Layer that modularizes key components of distributed learning, such as data distribution, model synchronization, communication protocols, and fault tolerance. Define standardized interfaces for each module to allow seamless integration and interchangeability of different implementations. Utilize a centralized coordinator or a decentralized consensus mechanism to manage task scheduling, resource allocation, and consistency across nodes. Incorporate middleware that handles network communication and data serialization, abstracting these complexities from the core learning algorithms. By providing these unified abstractions, developers can build, scale, and maintain distributed learning systems more efficiently without needing to manage underlying infrastructure details. |
| **Gemini 1.5 Flash Solution:** Implement a Federated Learning pattern. This pattern leverages a central server to coordinate model training across multiple decentralized clients (devices or nodes). Clients train local models on their own data, then send model updates (e.g., gradients) to the server. The server aggregates these updates to create a global model, which is then distributed back to the clients for further training. This addresses the lack of abstractions by providing a standardized framework for distributed model training, handling data privacy and communication complexities. |

Each of the 18 selected patterns was presented in an online survey form containing the original problem description and four candidate solutions (Literature, GPT-o1, Gemini, and Claude). Seven professionals with substantial software engineering experience participated. Five had > 10 years of experience, one between 6–10, and one 3–5 years. Regarding AI/ML expertise, three reported up to 2 years, two 3–5, and two 6–10 years. Most participants indicated strong familiarity with design patterns (five) and AI-based systems (four).

The solutions were anonymized and displayed in randomized order to avoid bias. Evaluators were then asked a single primary question: *Which solution best addresses the problem?* This procedure enabled a direct comparison of human preference between solutions documented in the literature and those generated by LLMs, providing evidence on the perceived quality and effectiveness of the different approaches. Table 6 summarizes the aggregated results.

**Table 5: Evaluation by specialists of solutions that exhibited low similarity to those discussed in the literature (rescaled 0–1)**

| Design Patterns for AI-Based Systems | Claude 3.5 Sonnet | o1-mini | Gemini 1.5 Flash | Solved (mean) |
|---|---|---|---|---|
| Parameter-Server Abstraction | 1.00 | 1.00 | 1.00 | **1.00** |
| Closed Loop intelligence | 1.00 | 1.00 | 1.00 | **1.00** |
| End-To-End Tests | 1.00 | 1.00 | 1.00 | **1.00** |
| Model-in-image pattern | 1.00 | 1.00 | 1.00 | **1.00** |
| Shadow | 1.00 | 1.00 | 1.00 | **1.00** |
| Legacy | 1.00 | 1.00 | 1.00 | **1.00** |
| Tar pit | 1.00 | 1.00 | 1.00 | **1.00** |
| Partial Rejection of Safety Responsibility | 0.67 | 1.00 | 1.00 | **0.89** |
| Player-Role Pattern | 0.67 | 1.00 | 1.00 | **0.89** |
| Giveaway | 1.00 | 1.00 | 0.67 | **0.89** |
| Model Published as Data | 1.00 | 0.67 | 1.00 | **0.89** |
| Full Acceptance of Safety Responsibility | 0.67 | 1.00 | 1.00 | **0.89** |
| Internal Feedback | 0.67 | 0.67 | 0.67 | **0.67** |
| Model Embedded in Application | 0.67 | 0.67 | 0.67 | **0.67** |
| Discard PoC code | 0.67 | 0.67 | 0.67 | **0.67** |
| Pipes and Filters | 0.67 | 0.67 | 0.67 | **0.67** |
| Kappa Architecture Pattern | 0.67 | 0.33 | 1.00 | **0.67** |
| Stateless Serving Function | 0.67 | 0.67 | 0.67 | **0.67** |
| Client-Server | 0.67 | 0.00 | 0.67 | **0.44** |
| Chop Shop | 0.33 | 0.33 | 0.67 | **0.44** |
| Decision Mode Switcher | 0.33 | 0.33 | 0.67 | **0.44** |
| Offline Predictions | 0.33 | 0.33 | 0.33 | **0.33** |
| AI Mode Switcher | 0.33 | 0.33 | 0.33 | **0.33** |
| Synchronous pattern | 0.00 | 0.00 | 0.00 | **0.00** |

**Table 6: Practitioner preferences in the blind evaluation.**

| Model | Votes | % | Rank |
|---|---|---|---|
| Claude 3.5 Sonnet | 42 | 33.3 % | 1 |
| Gemini 1.5 Flash | 36 | 28.6 % | 2 |
| o1-mini | 35 | 27.8 % | 3 |
| Literature | 13 | 10.3 % | 4 |

Practitioners clearly favored LLM-generated solutions, particularly those from Claude 3.5 Sonnet, which received more than triple the votes of the literature baseline. Interestingly, patterns with low textual similarity sometimes achieved high human preference, underscoring that novelty and clarity can enhance perceived quality even when solutions deviate from established formulations.

A more fine-grained analysis of the results at the pattern level, presented in Table 7, reveals a distribution of preferences that varies considerably across the 18 sampled design patterns. In several cases, the evaluators' votes resulted in ties, showing that no single approach consistently dominated. Nevertheless, Claude and o1-mini often emerged as the leading models, with Claude winning outright in patterns such as *Infrastructure as Code* and *Closed Loop Intelligence*, while o1-mini achieved the highest preference in *Prediction Cache Pattern* and *Tar Pit*. Gemini also obtained victories in patterns such as *Data Cache Pattern* and *Batch Serving*, though less frequently. Literature-based solutions were rarely the majority choice, with notable exceptions such as *Chop Shop* and cases of ties (*Decision Mode Switcher*, *Synchronous Pattern*), in which human-authored solutions still demonstrated competitive quality. These results indicate that LLMs are not universally superior but are highly competitive, often achieving or surpassing the perceived quality of literature-based solutions.

Taken together, these findings reinforce the potential of LLMs as design partners in software engineering for AI-based systems. The results show that LLMs can reproduce literature-based solutions and propose alternatives often judged equal or superior by humans. This opens new research directions, such as integrating LLMs into design processes as co-creators of patterns and exploring when their solutions are most likely to be trusted. The divergence between literature and LLM-generated solutions suggests a need to rethink how design knowledge is documented in the era of AI-assisted development.

## 5.4 RQ4 – Which LLM achieved the best overall performance?

To identify the best-performing model, results from all three dimensions were synthesized:

- **Similarity (RQ1):** o1-mini achieved the highest mean similarity with the literature (3.44).
- **Qualitative Judgment (RQ2):** Gemini 1.5 Flash provided the most valid alternatives among low-similarity cases.
- **Human Preference (RQ3):** Claude 3.5 Sonnet received the largest number of practitioner votes (42).

No single model dominated across all metrics. Rather, each exhibited a distinct strength: o1-mini aligned best with existing design knowledge, Gemini 1.5 Flash excelled in creative but functional

diversity, and Claude 3.5 Sonnet produced the most persuasive and well-structured solutions from a human perspective.

Overall, these findings show that modern LLMs can both replicate and expand upon human design reasoning, offering complementary capabilities for pattern-oriented AI system development.

# 6 Discussions

This section discusses the main findings of our study, organized around the four research questions. The discussion highlights the broader meaning of the results, identifies underlying patterns in model behavior, and connects them to existing knowledge in AI engineering and software design.

## 6.1 Interpretation of Results

**Similarity and Semantic Alignment (RQ1).** The results show that LLMs can reproduce established design solutions with moderate to high fidelity. The mean similarity scores (ranging between 3.31 and 3.44) suggest that these models have internalized substantial portions of software design knowledge, even without explicit fine-tuning on pattern repositories. This indicates that conceptual reasoning about system structure and component interaction can emerge from general purpose training. Notably, o1-mini's higher alignment with the literature reflects its optimization for concise, task-oriented reasoning, whereas Claude 3.5 Sonnet and Gemini 1.5 Flash exhibit slightly broader semantic interpretations that often lead to creative deviations.

**Expert Assessment of Divergent Solutions (RQ2).** Among the cases with low textual similarity ($\leq 3$), 75 % of the generated solutions were still judged as effective by experts. This finding is important: it shows that lack of surface similarity does not imply conceptual inaccuracy. LLMs often propose more abstract or generalized variants that remain faithful to the underlying intent of the original pattern. Such creativity mirrors human design diversity, in which different teams may produce distinct yet equally valid solutions for the same design problem.

**Human Perception and Preference (RQ3).** The blind evaluation revealed that practitioners often favored LLM-generated solutions over literature-based ones. Claude 3.5 Sonnet, for instance, received more than three times as many votes as the reference designs. This may reflect that LLMs naturally produce text that is concise, structured, and human-readable, which aligns with how practitioners prefer to consume design documentation. It also suggests that models can articulate pattern intent in ways that feel clearer or more actionable than original descriptions.

**Model-Specific Strengths (RQ4).** The comparison across models highlights complementary strengths rather than a single dominant performer. o1-mini most closely matched the literature, showing analytical precision; Gemini 1.5 Flash generated the broadest range of valid alternatives, evidencing creative flexibility; and Claude 3.5 Sonnet achieved the strongest acceptance among humans, indicating rhetorical and conceptual clarity. Together, these results suggest that ensembles or hybrid pipelines leveraging multiple LLMs could yield even better performance in design-oriented tasks.

## 6.2 Emerging Observations

Across all analyses, an interesting pattern emerges: LLMs tend to excel when design problems are framed at the architectural or component-interaction level, but struggle more with process-oriented or quality-assurance patterns. This is consistent with their training distribution—architectural reasoning is abundant in textual data (e.g., documentation, forums, and tutorials), whereas process-level reasoning requires contextual understanding of software life cycles. The slightly higher performance on "New" AI-specific patterns suggests that LLMs are naturally aligned with modern, data-driven design abstractions, which appear more frequently in their pre-training corpora than legacy object-oriented templates.

These findings collectively reinforce the idea that LLMs can act as *assistive reasoning agents* in the design of AI-based systems. They can recall canonical solutions, propose meaningful alternatives, and communicate design rationales in a human-readable form—all valuable traits in the early phases of system engineering.

## 6.3 Industrial and Research Implications

From an industrial perspective, the results open several opportunities for integrating LLMs into practical design workflows:

- **Design Exploration:** LLMs can serve as creative partners in design sessions, rapidly generating candidate solutions and helping teams compare alternatives before committing to implementation.
- **Documentation and Knowledge Transfer:** By expressing designs in natural language, models can automate the generation of pattern documentation and architectural rationales, improving communication between AI specialists and software engineers.
- **Educational Support:** The framework can be used in training environments to illustrate multiple correct ways of addressing the same design problem, encouraging critical analysis rather than rote memorization of patterns.

For researchers, this study contributes both methodologically and conceptually:

- It provides a **replicable framework** for evaluating design reasoning in LLMs, combining quantitative metrics and human perception.
- It demonstrates that **conceptual evaluation**—beyond code-level accuracy—is essential for understanding the true reasoning capabilities of generative models.
- It motivates further work on **hybrid human–AI co-design processes**, where LLMs assist in exploring, validating, and documenting architectural solutions.

Finally, these findings suggest that LLMs can complement human creativity rather than replace it. When embedded within a structured engineering process, they can accelerate idea generation, improve design communication, and help build a bridge between natural-language reasoning and formal architectural representation.

**Table 7: Per-pattern evaluation results with similarity categories and percentages**

| Design Pattern | Claude 3.5 Sonnet | Gemini 1.5 Flash | o1-mini | Literature | Category |
|---|---|---|---|---|---|
| Strategy Pattern | 1 | 2 | 4 | 0 | Similar |
| Data Lake (for ML) | 3 | 1 | 2 | 1 | Similar |
| Delegation of Safety Responsibility | 3 | 1 | 3 | 0 | Similar |
| Mediator Pattern | 1 | 3 | 3 | 0 | Similar |
| Infrastructure as Code | 4 | 3 | 0 | 0 | Similar |
| Multi-Layer Pattern | 3 | 2 | 1 | 1 | Similar |
| Data Cache Pattern | 2 | 5 | 0 | 0 | Similar |
| Dedicated Model API | 4 | 1 | 1 | 1 | Similar |
| Batch Serving | 1 | 4 | 1 | 1 | Similar |
| Distinguish Business Logic from ML Model | 2 | 3 | 1 | 1 | Similar |
| Prediction Cache Pattern | 1 | 1 | 5 | 0 | Similar |
| Tar pit | 1 | 2 | 3 | 1 | Not Similar |
| Chop Shop | 3 | 2 | 0 | 2 | Not Similar |
| Closed Loop Intelligence | 5 | 0 | 2 | 0 | Not Similar |
| Synchronous Pattern | 2 | 2 | 2 | 1 | Not Similar |
| Model Embedded in Application | 2 | 0 | 3 | 2 | Not Similar |
| Player-Role Pattern | 2 | 2 | 2 | 1 | Not Similar |
| Decision Mode Switcher | 2 | 2 | 2 | 1 | Not Similar |
| **Subtotal (Similar)** | **25 (32.5%)** | **26 (33.8%)** | **21 (27.3%)** | **5 (6.5%)** | Similar |
| **Subtotal (Not Similar)** | **17 (34.7%)** | **10 (20.4%)** | **14 (28.6%)** | **8 (16.3%)** | Not Similar |
| **Total** | **42 (33.3%)** | **36 (28.6%)** | **35 (27.8%)** | **13 (10.3%)** | – |

## 7 Threats to Validity

**Construct validity.** We defined evaluation dimensions based on established literature and adapted them to the context of LLM-generated design solutions. Nonetheless, the assessment of abstract qualities—such as efficiency or feasibility—may be subject to individual interpretation, potentially affecting scoring consistency. To address this, we engaged a diverse group of expert evaluators and employed a structured, scale-based scoring rubric to support more consistent judgments.

**Internal validity.** Human judgment was essential in evaluating the responses. While we selected evaluators with expertise in software engineering and AI-based systems, individual biases could still influence scores. To minimize such effects, we ensured consistent scoring instructions and used independent, double-blind evaluations.

**External validity.** The study was limited to three LLMs and two prompts focused on AI-based system design patterns. While we selected prompts to represent diverse and realistic scenarios, the results may not generalize to other software design contexts, such as low-level module design, non-pattern-oriented tasks, or domains outside AI-based systems. Moreover, our one-shot prompting strategy does not capture how practitioners might iteratively refine prompts or interact with LLMs in real-world settings, which may influence their performance in practice.

**Reliability.** LLMs exhibit probabilistic behaviour and may generate different responses for the same prompt. To ensure consistency, we fixed generation parameters and collected single responses per model per prompt. However, newer versions of these models may behave differently, and exact replication of outputs cannot be guaranteed. To support reproducibility, we provide all prompts, instructions, and model outputs.

## 8 Conclusion

This paper presented an empirical study on the ability of Large Language Models to reason about and generate design-pattern solutions for AI-based systems. Using a repository of 65 documented patterns and three state-of-the-art models (*Gemini 1.5 Flash*, *Claude 3.5 Sonnet*, and *o1-mini*), we evaluated model design capacity through a triangulated framework combining quantitative similarity analysis, expert judgment, and human preference evaluation.

The results demonstrate that LLMs can reproduce and even enhance human-authored design knowledge. Across all patterns, 90.76% of the generated solutions were equivalent or effective to those documented in the literature. These findings reinforce the idea that LLMs are not merely code generators but emerging partners in the design process—able to assist human creativity, accelerate design exploration, and promote a new paradigm of AI-supported software engineering.

As future work, we plan to extend this research by expanding the dataset with additional AI-engineering design repositories and industrial cases; (ii) incorporating automated semantic analysis and inter-rater agreement measures for higher statistical rigor and exploring interactive *human–LLM co-design environments*, where models collaborate with engineers in real time to generate, critique, and refine architectural solutions.

## Availability of Artifacts

The artifacts used in this work are available on https://github.com/feliperodrigues116/llms_identify_ai_design_patterns

## Acknowledgments

The authors thank the State University of Ceará (UECE), the GESAD, and the anonymous reviewers for their support and feedback.

## References


[1] J. Bogner, R. Verdecchia, and I. Gerostathopoulos. 2021. Characterizing Technical Debt and Antipatterns in AI-Based Systems: A Systematic Mapping Study. In *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)* (Madrid, Spain). 64–73. doi:10.1109/TechDebt52882.2021.00016

[2] Ljubisa Bojic, Olga Zagovora, Asta Zelenkauskaite, Vuk Vukovic, Milan Cabarkapa, Selma Veseljević Jerkovic, and Ana Jovančevic. 2025. Evaluating Large Language Models Against Human Annotators in Latent Content Analysis: Sentiment, Political Leaning, Emotional Intensity, and Sarcasm. In *arXiv preprint arXiv:2501.02532*.

[3] Jan Bosch, Helena Holmström Olsson, and Ivica Crnkovic. 2021. Engineering AI Systems: A Research Agenda. In *Artificial Intelligence Paradigms for Smart Cyber-Physical Systems*. 1–19.

[4] Tugba Celikten and Aytug Onan. 2025. Exploring Text Similarity in Human and AI-Generated Scientific Abstracts: A Comprehensive Analysis. *IEEE Access* 13 (2025), 74313–74334. doi:10.1109/ACCESS.2025.3564867

[5] William G. Cochran. 1977. *Sampling Techniques* (3rd ed.). John Wiley & Sons, New York.

[6] R. Damirchi and A. Amini. 2023. Non-Functional Requirement Extracting Methods for AI-based Systems: A Survey. In *2023 13th International Conference on Computer and Knowledge Engineering (ICCKE)* (Mashhad, Iran, Islamic Republic of). 535–539. doi:10.1109/ICCKE60553.2023.10326269

[7] A. Fan et al. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)* (Melbourne, Australia). 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008

[8] L. Heiland, M. Hauser, and J. Bogner. 2023. AI Patterns Repository. https://swe4ai.github.io/ai-patterns/. Accessed: July 6, 2025.

[9] L. Heiland, M. Hauser, and J. Bogner. 2023. Design Patterns for AI-based Systems: A Multivocal Literature Review and Pattern Repository. In *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)* (Melbourne, Australia). 184–196. doi:10.1109/CAIN58948.2023.00035

[10] Zoltán Richárd Jánki and Vilmos Bilicki. 2023. Rule-Based Architectural Design Pattern Recognition with GPT Models. *Electronics* 12, 15 (2023), 3364. doi:10.3390/electronics12153364

[11] Dae-Kyoo Kim. 2025. Quantitative Assessment of Generative Large Language Models on Design Pattern Application. *Computers, Materials & Continua* 82, 3 (2025), 3843–3872. doi:10.32604/cmc.2025.062552

[12] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, et al. 2022. Software Engineering for AI-based Systems: A Survey. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–59.

[13] Zhiqiang Pan, Hanqing Luo, Meng Li, and Hong Liu. 2024. Chain-of-action: Faithful and Multimodal Question Answering through Large Language Models. *arXiv preprint arXiv:2403.17359* (2024).

[14] Zhiqiang Pan, Xiaoyu Song, Yuwei Wang, Rui Cao, Bing Li, Yanan Li, and Hong Liu. 2025. Do Code LLMs Understand Design Patterns? *arXiv preprint arXiv:2501.04835* (2025).

[15] Sushant Kumar Pandey, Sivajeet Chand, Jennifer Horkoff, Miroslaw Staron, Miroslaw Ochodek, and Darko Durisic. 2025. Design Pattern Recognition: A Study of Large Language Models. *Empirical Software Engineering* 30, 3 (2025), 69. doi:10.1007/s10664-025-10625-1

[16] C. Schindler and A. Rausch. 2025. LLM-Based Design Pattern Detection. *arXiv preprint* abs/2502.18458 (2025).

[17] Hironori Washizaki, Foutse Khomh, Yann-Ga"el Gsuéhéneuc, Hideaki Takeuchi, Naoki Natori, Takashi Doi, and Shinji Okuda. 2022. Software Engineering Design Patterns for Machine Learning Applications. *Computer* 55, 3 (March 2022).

[18] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).